

University
Politehnica
of Bucharest





Cassandra (III)

Ciprian-Octavian Truică
ciprian.truica@upb.ro



Overview

- Installation
- Cassandra Query Language (CQL)
- CQL Data Types
- CQL Secondary Indexes
- CQL Aggregation
- CQL User Defined Functions (UDFs)



Installation

- To install Cassandra on a single node:
  - Download the archive

$ wget https://dlcdn.apache.org/cassandra/5.0.2/apache-cassandra-5.0.2-bin.tar.gz

  - Unzip the archive

$ tar -zxvf apache-cassandra-5.0.2-bin.tar.gz

  - Run Cassandra

$ cd ./apache-cassandra/bin
$ ./cassandra



Cluster Installation

- Installing Cassandra on a cluster (follow all the steps on each node)
  - Download the archive
  - Unzip the archive
  - Edit the cassandra.yaml ($CASSANDRA_HOME/conf/cassandra.yaml)
  - Start Cassandra on each node



Cluster Installation

- Edit the cassandra.yaml
  - cluster_name:
    - This is the name of your cluster.
  - - seeds:
    - This is a comma-delimited list of the IP address of each node in the cluster.
  - listen_address:
    - This is IP address that other nodes in the cluster will use to connect to this one.
    - It defaults to localhost and needs changed to the IP address of the node.



Cluster Installation

- Edit the cassandra.yaml
  - rpc_address:
    - This is the IP address for remote procedure calls.
    - It defaults to localhost.
    - If the server's hostname is properly configured, leave this as is.
    - Otherwise, change to server's IP address or the loopback address (127.0.0.1).
  - endpoint_snitch:
    - Name of the snitch, which is what tells Cassandra about what its network looks like.
    - This defaults to SimpleSnitch, which is used for networks in one datacenter.



Cluster Installation

- Edit the cassandra.yaml
  - auto_bootstrap:
    - This directive is not in the configuration file, so it has to be added and set to false.
    - This makes new nodes automatically use the right data.
    - It is optional if you're adding nodes to an existing cluster, but required when you're initializing a fresh cluster, that is, one with no data.



Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - SimpleSnitch:
    - Treats Strategy order as proximity.
    - This can improve cache locality when disabling read repair.
    - Only appropriate for single-datacenter deployments.
  - GossipingPropertyFileSnitch
    - This should be your go-to snitch for production use.
    - The rack and datacenter for the local node are defined in cassandra-rackdc.properties and propagated to other nodes via gossip.
    - If cassandra-topology.properties exists, it is used as a fallback, allowing migration from the PropertyFileSnitch.



Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - PropertyFileSnitch:
    - Proximity is determined by rack and data center, which are explicitly configured in cassandra-topology.properties.
  - Ec2Snitch:
    - Appropriate for EC2 deployments in a single Region.
    - Loads Region and Availability Zone information from the EC2 API.
    - The Region is treated as the datacenter, and the Availability Zone as the rack.
    - Only private IPs are used, so this will not work across multiple Regions.



Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - Ec2MultiRegionSnitch:
    - Uses public IPs as broadcast_address to allow cross-region connectivity.
    - Thus, you should set seed addresses to the public IP as well.
    - You will need to open the storage_port or ssl_storage_port on the public IP firewall.
    - For intra-Region traffic, Cassandra will switch to the private IP after establishing a connection.



Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - RackInferringSnitch:
    - Proximity is determined by rack and data center, which are assumed to correspond to the 3rd and 2nd octet of each node's IP address, respectively.
    - Unless this happens to match your deployment conventions, this is best used as an example of writing a custom Snitch class and is provided in that spirit.

Cluster Installation

```
# The name of the cluster. This is mainly used to prevent machines in
# one logical cluster from joining another.
cluster_name: 'DBCassandraCluster'

...

# any class that implements the SeedProvider interface and has a
# constructor that takes a Map<String, String> of parameters will do.
seed_provider:
    # Addresses of hosts that are deemed contact points.
    # Cassandra nodes use this list of hosts to find each other and learn
    # the topology of the ring.  You must change this if you are running
    # multiple nodes!
    - class_name: org.apache.cassandra.locator.SimpleSeedProvider
      parameters:
          # seeds is actually a comma-delimited list of addresses.
          # Ex: "<ip1>,<ip2>,<ip3>"
          - seeds: "172.19.3.31,172.19.3.32,172.193.33"
```



Cluster Installation

```
# Address or interface to bind to and tell other Cassandra nodes to connect to.
# You _must_ change this if you want multiple nodes to be able to communicate!
#
# Set listen_address OR listen_interface, not both.
#
# Leaving it blank leaves it up to InetAddress.getLocalHost(). This
# will always do the Right Thing _if_ the node is properly configured
# (hostname, name resolution, etc), and the Right Thing is to use the
# address associated with the hostname (it might not be).
#
# Setting listen_address to 0.0.0.0 is always wrong.
#
listen_address: 172.19.3.31

...

# The address or interface to bind the Thrift RPC service and native transport
# server to.
#
# Set rpc_address OR rpc_interface, not both.
#
# Leaving rpc_address blank has the same effect as on listen_address
# (i.e. it will be based on the configured hostname of the node).
#
# Note that unlike listen_address, you can specify 0.0.0.0, but you must also
# set broadcast_rpc_address to a value other than 0.0.0.0.
#
# For security reasons, you should not expose this port to the internet.  Firewall it if needed.
rpc_address: 172.19.3.31
```



Cluster Installation

```
# You can use a custom Snitch by setting this to the full class name
# of the snitch, which will be assumed to be on your classpath.
endpoint_snitch: GossipingPropertyFileSnitch


auto_bootstrap: false
```

Cluster Installation

- To verify the status of you cluster use nodetool utility.

$ nodetool status

```
Datacenter: dc1
===============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address       Load       Tokens       Owns (effective)  Host ID                               Rack
UN  172.19.3.33   321.07 KiB  256          100.0%            26e4ec87-b562-448e-af70-7e62dba50abe  rack1
UN  172.19.3.32   348.49 KiB  256          100.0%            90a71c7a-15ac-41b8-8437-78984d89b7e4  rack1
UN  172.19.3.31   322.63 KiB  256          100.0%            89cec698-d686-4d97-9024-088babd9ebc3  rack1
```



CQL

- Cassandra Query Language (CQL)
- The syntax of CQL is similar in many ways to SQL, but with some important differences.
- To start the CLI use cqlsh ($CASSANDRA_HOME/bin/cqlsh)
- The default port for cqlsh is 9042
- If the server is listening on localhost then issue the following command:

```
$ cqlsh
```

- If the server is listening on a different IP/PORT (used by one of the nodes in the cluster), use

```
$ cqlsh IP PORT
```



CQL

- To see all the commands use HELP
- See the cluster details

```
cqlsh> DESCRIBE CLUSTER;
```

- To see the keyspaces available:

```
cqlsh> DESCRIBE KEYSPACES;
```

- Initially this list will consist of several system keyspaces.
- Once you have created your own keyspaces, they will be shown as well.
- The system keyspaces are managed internally by Cassandra.
- To see the client, server, and protocol versions:

```
cqlsh> SHOW VERSION;
```



CQL
Creating a Keyspace

- When creating a keyspace, there are different strategies to choose from:

1. SimpleStrategy
   - Places replicas at consecutive nodes around the ring, starting with the node indicated by the partitioner.
   - The replication factor must be specified

2. NetworkTopologyStrategy
   - Allows you to specify a different replication factor for each data center

3. OldNetworkTopologyStrategy - deprecated staring with v4
   - Is similar to the NetworkTopologyStrategy in that it places replicas in multiple data centers, but its algorithm is less sophisticated.
   - This is depricate, it was keept just for backwards compatibility.



CQL

- The following command creats a Keyspace

```
cqlsh> CREATE KEYSPACE faculty WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 3 };
```

- To bypass writing to the commit log for the keyspace then you can use durable_writes property.

```
cqlsh> CREATE KEYSPACE faculty WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 3 } AND durable_writes = true;
```

- To delete a Keyspace use the DROP command

```
cqlsh> DROP KEYSPACE faculty;
```

- To use a keyspace and start creating tables:

```
cqlsh> USE faculty
```



CQL

- To creating a table, use the CREATE TABLE command

```
cqlsh:faculty> CREATE TABLE TEACHERS(
                    id int PRIMARY KEY,
                    firstname VARCHAR,
                    lastname VARCHAR,
                    age INT );
```

- Tables must have a primary key upon creation
- To modify a table, use ALTER TABLE command
- To drop a table, use DROP TABLE command

```
cqlsh:faculty> DROP TABLE teachers;
```



CQL

- Adding data to a table use the INSERT command
- The columns need to be specified

```
cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(1, 'Ciprian-Octavian', 'Truică', 32);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(2, 'Florin', 'Rădulescu', 58);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(3, 'Alexandru', 'Boicea', 59);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(4, 'Sorin', 'Ciolofan', 37);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(5, 'Dan', 'Schrager', 59);
```



CQL

- Selection and projection is done using the SELECT command

```
cqlsh:faculty> SELECT * FROM teachers;

cqlsh:faculty> SELECT firstname, lastname FROM teachers
WHERE id IN (1, 3);
```

- When filtering we need to specify that we are filtering the data using ALLOW FILTERING property

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers
WHERE age <= 40 ALLOW FILTERING;
```



CQL

- Some limitations on SELECT command:
  - There are no nested queries
  - Some operators don't work, e.g., between ... and ... ,
  - Some operators work only on primary keys, e.g., IN.
  - There are no JOIN operations



CQL

- Update a value

```
cqlsh:faculty> UPDATE teachers SET lastname = 'Truică' WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET lastname = 'Truică', firstname = 'Ciprian'
WHERE id = 1;
```

- Delete a value

```
cqlsh:faculty> DELETE FROM teachers WHERE id = 5;
```

- Delete all values

```
cqlsh:faculty> TRUNCATE teachers;
```



CQL Data Types

- CQL Data Types
  - Numeric Data Types
  - Textual Data Types
  - Time and Identity Data Types
  - Other Simple Data Types
  - Collections
  - User-Defined Types



CQL Data Types
Numeric Data Types

- Integer and floating-point numbers.

- These types are similar to standard types in Java:
  - int - A 32-bit signed integer (as in Java)
  - bigint - A 64-bit signed long integer (equivalent to a Java long)
  - smallint - A 16-bit signed integer (equivalent to a Java short)
  - tinyint - An 8-bit signed integer (as in Java)
  - varint - A variable precision signed integer (equivalent to java.math.BigInteger)
  - float - A 32-bit IEEE-754 floating point (as in Java)
  - double - A 64-bit IEEE-754 floating point (as in Java)
  - decimal - A variable precision decimal (equivalent to java.math.BigDecimal)



CQL Data Types
Textual Data Types

- CQL provides two data types for representing text:
  - text, varchar - Synonyms for a UTF-8 character string
  - ascii - An ASCII character string

- The ascii type is most useful if you are dealing with legacy data that is in ASCII format.



CQL Data Types
Time and Identity Data Types

- The identity of data elements such as rows and partitions is important in any data model in order to be able to access the data.

- Cassandra provides several types which prove quite useful in defining unique partition keys.

- Time Data Types:
  - timestamp
  - date and time
  - uuid
  - timeuuid



CQL Data Types
Time and Identity Data Types

- timestamp
  - Each column has a timestamp indicating when it was last modified
  - A timestamp can also be used as the value of a column itself
  - The time can be encoded as a 64-bit signed integer
  - It is typically much more useful to input a timestamp using one of several supported ISO 8601 date formats
  - The best practice is to always provide time zones rather than relying on the operating system time zone configuration



CQL Data Types
Time and Identity Data Types

- date, time
  - Releases through Cassandra 2.1 only had the timestamp type to represent times, which included both a date and a time of day.

  - The 2.2 release introduced date and time types that allowed these to be represented independently, that is:
    - A date without a time
    - A time of day without reference to a specific date.



CQL Data Types
Time and Identity Data Types

- uuid
  - Universally unique identifier (UUID) is a 128-bit value in which the bits conform to one of several types, of which the most commonly used are known as Type 1 and Type 4.
  - The CQL uuid type is a Type 4 UUID, which is based entirely on random numbers.
  - UUIDs are typically represented as dash-separated sequences of hex digits.
  - The uuid type is often used as a surrogate key, either by itself or in combination with other values.
  - You can obtain a Type 4 UUID value via the uuid() function and use this value in an INSERT or UPDATE.



CQL Data Types
Time and Identity Data Types

- timeuuid
  - This is a Type 1 UUID, which is based on the following 3 components:
    - The MAC address of the computer
    - The system time
    - A sequence number used to prevent duplicates
  - This type is frequently used as a conflict-free timestamp
  - cqlsh provides several convenience functions for interacting with the timeuuid type:
    - now()
    - dateOf()
    - unixTimestampOf().



CQL Data Types
Other Simple Data Types

- boolean
  - This is a simple true/false value.
  - The cqlsh is case insensitive in accepting these values but outputs True or False

- inet
  - This type represents IPv4 or IPv6 Internet addresses.
  - cqlsh accepts any legal format for defining IPv4 addresses, including:
    - Dotted or non-dotted representations containing:
      - Decimal values.
      - Octal values.
      - Hexadecimal values.
  - IPv6 addresses are represented as eight groups of four hexadecimal digits, separated by colons



CQL Data Types
Other Simple Data Types

- counter
  - The counter data type provides 64-bit signed integer
  - Its value cannot be set directly, but only incremented or decremented.
  - Cassandra is one of the few databases that provides race-free increments across data centers
  - Counters are frequently used for tracking statistics such as numbers of page views, tweets, log messages, and so on.
  - The counter type has some special restrictions: It cannot be used as part of a primary key.



CQL Data Types
Other Simple Data Types

- counter
  - The increment and decrement operators are not idempotent.
  - There is no operation to reset a counter directly, but you can approximate a reset by reading the counter value and decrementing by that value.
  - Unfortunately, this is not guaranteed to work perfectly, as the counter may have been changed elsewhere in between reading and writing.



CQL Data Types

- CQL Collections
  - Set
  - List
  - Map



CQL Data Types
Set

- The set data type stores a collection of elements
- The elements are unordered, but cqlsh returns the elements in sorted order
- Sets can contain:
    - Simple types
    - User-defined types
    - Other collections
- One advantage of using set is the ability to insert additional items without having to read the contents first



CQL Data Types
Set

```
cqlsh:faculty> ALTER TABLE teachers ADD offices SET<TEXT>;

cqlsh:faculty> UPDATE teachers SET offices= {'EF208', 'PR305', 'EG105'} WHERE id=1;

cqlsh:faculty> UPDATE teachers SET offices= {'Vice Dean Office'} WHERE id=2;

cqlsh:faculty> UPDATE teachers SET offices= {'EF309'} WHERE id=3;

cqlsh:faculty> UPDATE teachers SET offices= {'EF209'} WHERE id=4;

cqlsh:faculty> UPDATE teachers SET offices= {'PR304'} WHERE id=5;
```



CQL Data Types
Set operations

- Adding a new element using +

```
cqlsh:faculty> UPDATE teachers SET offices= offices + {'EF309'} WHERE id =2;
```

- Removing an element using -

```
cqlsh:faculty> UPDATE teachers SET offices= offices - {'EG105'} WHERE id =1;
```

- Empty the set use {}

```
cqlsh:faculty> UPDATE teachers SET offices= { } WHERE id =4;
```

- Select

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE offices
CONTAINS 'EF309' ALLOW FILTERING;
```



CQL Data Types
List

- The list data type contains an unordered list of elements.
- By default, the values are stored in order of insertion
- Uses the same operations as sets to add and remove an element
- The empty a list is []
- Indexes can be used to access elements (in the WHERE clause of SELECT and SET clause of UPDATE).
- An element can be deleted using the index.



CQL Data Types List

```
cqlsh:faculty> ALTER TABLE teachers ADD telephone list<text>;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.21', '+40
721.222.111'] WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.22', '+40
721.222.112'] WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.23', '+40
721.222.113'] WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.23', '+40
721.222.114'] WHERE id = 4;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.25', '+40
721.222.115'] WHERE id = 5;
```



CQL Data Types List

```
cqlsh:faculty> UPDATE teachers SET telephone = telephone - ['+40 721.222.111']
WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET telephone = telephone + ['+40 21 323.222.30']
WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET telephone = [ ] WHERE id = 5;

cqlsh:faculty> UPDATE teachers SET telephone[0] = '+40 21 323.222.90' WHERE id = 2;

cqlsh:faculty> DELETE telephone[2] FROM teachers  WHERE id = 3;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE telephone
CONTAINS '+40 21 323.222.23' ALLOW FILTERING;
```



CQL Data Types
Map

- The map data type contains a collection of key/value pairs
- The keys and the values can be of any type except counter
- Collection types are very useful in cases where we need to store a variable number of elements within a single column



CQL Data Types
Map

```
cqlsh:faculty> ALTER TABLE teachers ADD title map<text,text>;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'teaching assistant',
'profession': 'engineer'} WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'professor',
'profession': 'engineer'} WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'associate professor',
'profession': 'engineer'} WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'lecturer', 'profession':
'engineer'} WHERE id = 4;
```



CQL Data Types
Map

```
cqlsh:faculty> UPDATE teachers SET title = {'academic': 'lecturer',
'profession': 'engineer'} WHERE id = 5;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE
title['academic'] = 'teaching assistant' ALLOW FILTERING;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title
contains 'lecturer' ALLOW FILTERING;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title
CONTAINS KEY 'academic' ALLOW FILTERING;
```



CQL Data Types
User Defined Types (UDT)

- Cassandra supports UDTs
- They can be created using CREATE TYPE command
- They can be used on their own or using a collection, where they need to be frozen

```
cqlsh:faculty> CREATE TYPE course(
                    name VARCHAR,
                    year INT
                );
```



CQL Data Types
UDT

```
cqlsh:faculty> ALTER TABLE teachers ADD courses list<frozen<course>>;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'DM', year: 5}] WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'UBD', year: 4}, {name: 'DMDW', year: 5}] WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'SCAD', year: 4}, {name: 'ICASE', year: 5}] WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'PC', year: 1}, {name: 'ABD',
year: 5}] WHERE id = 4;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'PAJ', year: 5}, {name: 'MN',
year: 1}] WHERE id = 5;
```



CQL Secondary Indexes

- To create a secondary index use the CREATE INDEX command
- It's also possible to create indexes that are based on values in collections.
- For optimal read performance: denormalized table designs or materialized views are generally preferred to using secondary indexes.
- Note that for maps in particular, we have the option of indexing either the keys (via the syntax KEYS(addresses)) or the values (which is the default), or both (in Cassandra 2.2 or later).



CQL Secondary Indexes

```
cqlsh:faculty> CREATE INDEX idx_offices ON teachers(offices);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE offices
CONTAINS 'EF309';

cqlsh:faculty> DROP INDEX idx_offices;



cqlsh:faculty> CREATE INDEX idx_telephone ON teachers(telephone);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE
telephone CONTAINS '+40 21 323.222.23';

cqlsh:faculty> DROP INDEX idx_telephone;
```



CQL Secondary Indexes

```
cqlsh:faculty> CREATE INDEX idx_title ON teachers(title);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title CONTAINS
'lecturer';

cqlsh:faculty> DROP INDEX idx_title;



cqlsh:faculty> CREATE INDEX idx_title_key ON teachers(keys(title));

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title CONTAINS KEY
'academic' ALLOW FILTERING;

cqlsh:faculty> DROP INDEX idx_title_key;
```



CQL Secondary Indexes

- SASI: A New Secondary Index Implementation
  - The Cassandra 2.4 release included an alternative implementation of secondary indexes known as the SSTable Attached Secondary Index (SASI).
  - SASI was developed by Apple and released as an open source implementation of Cassandra’s secondary index API.
  - As the name implies, SASI indexes are calculated and stored as part of each SSTable file, differing from the original Cassandra implementation, which stores indexes in separate, "hidden" tables.
  - Enable it in Cassandra.yaml: sasi_indexes_enabled: false



CQL Secondary Indexes

- SASI: A New Secondary Index Implementation
  - The SASI implementation exists alongside traditional secondary indexes.
  - SASI indexes do offer functionality beyond the traditional secondary index implementation, such as the ability to do inequality (greater than or less than) searches on indexed columns.
  - You can also use the new CQL LIKE keyword to do text searches against indexed columns.
  - While SASI indexes do perform better than traditional indexes by eliminating the need to read from additional tables, they still require reads from a greater number of nodes than a denormalized design.



CQL Secondary Indexes

- SASI: A New Secondary Index Implementation

```
cqlsh:faculty> CREATE CUSTOM index idx_lastname_sasi ON
teachers(lastname) USING 'org.apache.cassandra.index.sasi.SASIIndex';


cqlsh:faculty> select firstname, lastname from teachers where lastname like
'Tr%';

cqlsh:faculty> drop index idx_lastname_sasi;
```



CQL Aggregation

- Built-in functions and aggregates
  - COUNT
    - The COUNT function is used to count the number of rows returned by a query.
    - This command can also be used to count the number of non-null values of a specified column.
  - MIN and MAX
    - The MIN and MAX functions can be used to compute the minimum and maximum value returned by a query for a given column.



CQL Aggregation

- Built-in functions and aggregates
  - SUM
    - The sum function can be used to sum up all the values returned by a query for a given column.
  - AVG
    - The avg function can be used to compute the average of all the values returned by a query for a given column.



CQL Aggregation

- Built-in functions and aggregates

```
cqlsh:faculty> SELECT COUNT(firstname) FROM teachers;

cqlsh:faculty> SELECT MIN(age), MAX(age) FROM teachers;

cqlsh:faculty> SELECT SUM(age) FROM teachers;

cqlsh:faculty> SELECT AVG(age) FROM teachers;
```



CQL UDFs

- User defined functions
  - UDFs are functions that are applied on Cassandra nodes to stored data as part of query processing.
  - Before using UDFs in your cluster, they must be enabled in the cassandra.yaml file on each node:
    - Before v4: enable_user_defined_functions: true
    - Starting with v4: user_defined_functions_enabled: true
  - The 3.0 release added a security feature to run UDF code in a separate sandbox to limit the ability for a malicious function to gain unauthorized access to a node's Java runtime.



CQL UDFs

- UDF Restrictions
  - The code you can write inside a UDF is quite limited
  - It has to be a pure function so no IO, logging, external access of any kind.
  - Can't start threads
  - No dependencies



CQL UDFs

- User defined functions

```
cqlsh:faculty> CREATE FUNCTION age_to_range(input int)
           ... RETURNS NULL ON NULL INPUT
           ... RETURNS varchar
           ... LANGUAGE java AS '
           ...    if (input <= 35) return "young";
           ...    else if (input> 55) return "old";
           ...    else return "mature";
           ... ';

cqlsh:faculty> SELECT firstname, lastname, age_to_range(age) FROM
teachers;
```



CQL UDFs

- Custom aggregates allow you to reduce a query that returns many rows into a single value
- These are called user defined aggregation (UDA)
- For UDAs we need to:
  - Define a function that will be executed, sequentially, for every row, passing some state along
  - Initial state for the first row
  - An optional function that will take in the final state and produce a value



CQL UDFs

- Custom aggregates State function – average:

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION avgState ( state
tuple<int,bigint>, val int ) CALLED
        ... ON NULL INPUT RETURNS tuple<int,bigint> LANGUAGE java AS '
        ...   if (val != null) {
        ...     state.setInt(0, state.getInt(0)+1);
        ...     state.setLong(1,state.getLong(1)+val.intValue());
        ...   }
        ...   return state;
        ... ';
```



CQL UDFs

- Custom aggregates Final function – average:

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION avgFinal ( state
tuple<int,bigint> ) CALLED ON
        ... NULL INPUT RETURNS double LANGUAGE java AS '
        ...   double r = 0;
        ...   if (state.getInt(0) == 0)
        ...     return null;
        ...   r = state.getLong(1);
        ...   r /= state.getInt(0);
        ...   return Double.valueOf(r);
        ... ';
```



CQL UDFs

- Custom aggregates creating the aggregate – average

```
cqlsh:faculty> CREATE AGGREGATE IF NOT EXISTS average ( int )
           ... SFUNC avgState
           ... STYPE tuple<int,bigint>
           ... FINALFUNC avgFinal
           ... INITCOND (0,0);

cqlsh:faculty> SELECT average(age) FROM teachers;
```



CQL UDFs

- Custom aggregation to simulate – unique

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION uniq(state set<int>, val int)
           ... CALLED ON NULL INPUT RETURNS set<int> LANGUAGE java
           ... AS 'state.add(val); return state;';

cqlsh:faculty> CREATE OR REPLACE AGGREGATE uniq(int)
           ... SFUNC uniq STYPE set<int> INITCOND {};


cqlsh:faculty> SELECT uniq(age) FROM teachers;
```



CQL UDFs

- Custom aggregates simulate group by

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION state_group_and_max(state map<int, int>, type
int, amount int)
           ... CALLED ON NULL INPUT
           ... RETURNS map<int, int>
           ... LANGUAGE java AS '
           ...   Integer val = (Integer) state.get(type);
           ...   if (val == null) val = amount;
           ...   else val = Math.max(val, amount);
           ...   state.put(type, val);
           ...   return state;
           ... ';

cqlsh:faculty> CREATE OR REPLACE AGGREGATE state_group_and_max(int, int)
           ... SFUNC state_group_and_max
           ... STYPE map<int, int>
           ... INITCOND {};

cqlsh:faculty> SELECT state_group_and_max(age, id) FROM teachers;
```



Bibliography

- Apache Cassandra Documentation https://cassandra.apache.org/doc/latest/

- Jeff Carpenter, Eben Hewitt. Cassandra: The Definitive Guide: Distributed Data at Web Scale. O'Reilly Media, Inc., 2016.

- Christof Strauch. "NoSQL databases." *Lecture Notes, Stuttgart Media University* 20.24 (2011): 79.

- Eric Redmond, Jim Wilson. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 1nd Edition 2012

- Luc Perkins, Jim Wilson, and Eric Redmond. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2nd Edition 2018